

FixTime — Plataforma de Agendamento

Gestão Operacional de Visitas Técnicas com SDD (Spec-Driven Development), Governança GitHub e Test Harness Automatizado

Status Geral da Entrega: O projeto FixTime atende integralmente aos 4 pilares da avaliação: Repositório com Governança (zero commits diretos na main, PRs com revisão cruzada, board Kanban), Especificação Técnica SDD canônica em docs/SPEC.md (RFs, RNFs mensuráveis, RN01 a RN08, contratos JSON), Ambiente Dockerizado 100% reproduzível com copilotos de IA documentados, e Test Harness com **104 testes automatizados em verde** (44 backend + 60 frontend) com rastreabilidade formal via docstrings.

Integrantes da Equipe:

Caio Boudens de Castro RA: 22304522	Eduardo Frois Drumond RA: 22303035
Fernando Medeiros Farias RA: 22306100	Larissa Queiroz Ramos RA: 22304308
Mayssa Barbosa Dias RA: 22303603	Thiago Venâncio Gomides RA: 22307398

104 TESTES APROVADOS	100% COBERTURA REGRAS	0 COMMITTS DIRETOS MAIN	1 CMD DOCKER COMPOSE UP
--------------------------------	---------------------------------	-----------------------------------	-----------------------------------

1. Repositório GitHub com Governança

A governança do projeto FixTime adota padrões rigorosos de engenharia de software colaborativa, rastreabilidade e proteção de branches:

Requisito	Status	Evidência e Implementação no Repositório
Estratégia de Branches	CONFORME	Estrutura composta por main (release estável), develop (integração contínua da sprint) e branches isoladas de feature nomeadas por issue (ex: feature/issue-1-contrato-agendamento, feature/issue-28-exportacao-csv).
Proteção de Branches	CONFORME	Regras de proteção ativas no GitHub exigindo abertura obrigatória de Pull Request, revisão com aprovação de outro membro e execução com sucesso da suíte de CI.
Zero Commits na Main	CONFORME	Nenhum commit direto foi realizado na branch main. Todo código entra exclusivamente via Pull Requests aprovados.
GitHub Projects & Issues	CONFORME	Quadro Kanban do projeto FixTime organizado com Issues detalhadas, critérios de aceite, labels de prioridade/épicas e responsáveis (assignees) atribuídos.
Revisão Cruzada em PRs	CONFORME	Histórico com dezenas de PRs (PR #24 a #60) revisados e aprovados por integrantes diferentes dos desenvolvedores que submeteram o código.
README com ADRs	CONFORME	README completo com visão geral, equipe, comandos de execução, guia de testes e Registro de Decisões Arquiteturais (ADR-001, ADR-002, ADR-003).

Síntese das Decisões Arquiteturais (ADRs):

- **ADR-001 (Spring Boot & React):** Spring Boot 3.4.5 para APIs REST corporativas com validação Bean Validation; React 19 com TypeScript e Vite para velocidade de renderização e tipagem no frontend.
- **ADR-002 (Regras de Negócio no Service):** Todas as validações de jornada, antecedência, conflitos de horário e feriados residem exclusivamente na camada de serviço (AppointmentService), mantendo controllers focados apenas no contrato HTTP.
- **ADR-003 (PostgreSQL no Docker e H2 nos Testes):** PostgreSQL 16 para o ambiente integrado de execução; banco em memória H2 nos testes para garantir rapidez, isolamento e independência de infraestrutura.

2. Especificação Técnica (SDD) — em docs/SPEC.md

A especificação canônica do sistema encontra-se no arquivo docs/SPEC.md, documentando o domínio e formalizando requisitos e contratos:

2.1. Problema Real e Personas: Pequenas assistências técnicas enfrentam perda de receita e sobreposição de horários ao controlar atendimentos por planilhas e ligações. O FixTime atende ao *Operador* (gestão e conciliação), ao *Técnico* (jornada de campo) e ao *Cliente* (solicitação pontual).

2.2. Requisitos Funcionais (RF) e Não-Funcionais Mensuráveis (RNF):

Código	Requisito	Critério de Aceite / Métrica Quantificável
RF01 - RF03	CRUDs Clientes, Técnicos e Serviços	Cadastro ativo, listagem e validação de campos obrigatórios.
RF04	Consulta de Disponibilidade	Retorna janelas livres em dias úteis; lista vazia em feriados/bloqueios.
RF05	Criação e Listagem de Agendamentos	Cálculo automático de endsAt, paginação e filtros combinados.
RF06 - RF07	Cancelamento e Conclusão de Visita	Endpoints PATCH com validações de antecedência ($\geq 2h$) e término.
RF08	Exportação em Streaming CSV	Exportação RFC 4180 com codificação UTF-8 BOM para planilhas.
RNF01 - RNF02	API REST e Desempenho	Prefixo /api/v1 e latência transacional P95 < 300ms .
RNF03 - RNF04	Erros Estruturados e Cobertura	JSON padronizado ErrorResponse; cobertura de testes > 85% .
RNF05	Reprodutibilidade Docker	Inicialização fullstack com único comando docker compose up.

2.3. Regras de Negócio Numeradas (RN):

- **RN01:** Cliente, técnico e serviço devem existir no banco e estar ativos (active = true).
- **RN02:** Duração do serviço positiva (> 0 min) e preço não-negativo (>= 0.00). Término (endsAt) calculado pelo backend.
- **RN03:** Início da visita deve ter pelo menos **2 horas de antecedência** em relação ao momento da requisição.
- **RN04:** Visitas ocorrem exclusivamente em **dias úteis das 08:00 às 18:00**, terminando no mesmo dia civil.
- **RN05:** Bloqueio de sobreposição de horário para o mesmo técnico (intervalos adjacentes são permitidos).
- **RN06:** Cancelamento permitido apenas em status SCHEDULED e com antecedência >= 2 horas.
- **RN07:** Conclusão permitida apenas em status SCHEDULED e estritamente após o horário de término (endsAt).
- **RN08:** Bloqueio automático de agendamentos em feriados nacionais brasileiros e datas bloqueadas cadastradas.

3. Agente de IA + Ambiente Padronizado

O projeto padronizou o uso de copilotos de Inteligência Artificial e a infraestrutura de contêineres para máxima reprodutibilidade:

3.1. Governança e Arquivos de Contexto de IA Versionados:

- AGENTS.md: Diretrizes obrigatórias de governança, convenções de código Spring Boot/React e fluxo SDD.
- .cursorrules: Contexto arquitetural e regras de validação para desenvolvimento no editor Cursor.
- .github/copilot-instructions.md: Instruções de alinhamento com a especificação técnica para o GitHub Copilot.
- **Rastreabilidade em PRs:** Todas as contribuições com suporte de IA foram revisadas, testadas e validadas por membros humanos da equipe antes de qualquer aprovação.

3.2. Reprodutibilidade 100% Docker (Zero Configuração Prévia):

A aplicação é completamente reprodutível. Qualquer usuário pode clonar o repositório e executar o ecossistema fullstack com um único comando, sem necessidade de Java, Maven ou Node.js locais:

Comando Único de Execução do Ambiente Completo:

docker compose up --build

Serviço	Imagem / Base	Porta	Função no Ecossistema FixTime
database	postgres:16-alpine	5432	Banco relacional com healthcheck ativo para integridade.
backend	Eclipse Temurin 21 JRE	8080	API REST Spring Boot com JPA, DTOs e validações de serviço.
frontend	Nginx Alpine (Multi-stage)	5173 / 80	SPA React com proxy reverso automático para backend:8080.

4. Test Harness (Suíte de Testes Automatizados)

A suíte automatizada do FixTime é composta por **104 testes em verde**, executáveis via comando único no Docker ou localmente:

Comando Único de Execução do Test Harness via Docker:

docker compose run --rm backend mvn test

Componente de Teste	Tecnologia	Testes	Status	Rastreabilidade e Regras Validadas
AppointmentServiceTest	JUnit 5 / Mockito	19	PASS	RN01 a RN08 com Clock fixo (criação, antecedência, feriados).
AppointmentIntegrationTest	MockMvc / H2	11	PASS	Fluxo completo REST, paginação, filtros e resposta estruturada 400.
BlockedDateIntegrationTest	MockMvc / H2	5	PASS	RN08: Bloqueio de datas, feriados nacionais e rejeição 409.
AppointmentExportTest	MockMvc / Unit	5	PASS	RF08: Conformidade RFC 4180 e UTF-8 BOM para streaming CSV.
Customer & Health Tests	JUnit 5 / WebMvc	4	PASS	RF01/RN01 (ativação cadastral) e RNF01 (endpoint /api/v1/health).
Frontend Test Suite	Vitest / RTL	60	PASS	8 arquivos cobrindo páginas, modais, elegibilidade e cliente HTTP.
TOTAL CONSOLIDADO	Fullstack	104	100% OK	Zero falhas, zero erros e zero testes ignorados.

4.2. Rastreabilidade Formal via Docstrings: Cada método de teste no backend e frontend inclui Javadoc ou JSDoc explícito mapeando o teste à respectiva regra ou contrato (ex.: `/** Valida RN05 e RF05: Bloqueio de sobreposição */`). O relatório completo encontra-se em docs/test-report.md.